SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Industrial Problem Solving

COURSE CODE: CSA0904 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO4: *Applet Programming*-Applet Architecture - Simple Applet Display Methods Event handling in Java - Delegation Event Model - Event Classes - Sources of Events - Event Listener Interfaces -AWT Component and GUI Design : AWT Classes, Controls, Layout Managers, and Menus.*(BL3,BL4)*
Assessment Tool Weightage: 20%

QUESTIONS: 2 **Total Marks: 100**

ANNEXURE A

Industrial Problem Solving

Code of Conduct:
*I **O. Bhanu Prakash Reddy-192372075** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: obhanuprakashreddy

RUBRICS FOR CALCULATION:

Criteria	**Wt. %**	**Level 4 Exemplary** 100% of weightage	**Level 3 Proficient** 75% of weightage	**Level 2 Developing** 50% of weightage	**Level 1 Beginning** 25% of weightage
Problem Understanding	**20%**	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	**25%**	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	**20%**	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	**20%**	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	**15%**	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases
Total	**100 %**				

1. **Hospital Appointment Management System**
   Design a Swing-based GUI application for a hospital to manage patient appointments, including booking, updating, and canceling schedules. Use AWT/Swing controls such as buttons, text fields, tables, and menus to facilitate user interaction. Implement event listeners to handle actions like appointment confirmation and deletion. Ensure the system can handle dynamic patient data efficiently. Evaluate different layout managers (FlowLayout, GridLayout, BorderLayout) for organizing complex UI components. Analyze how the design supports scalability when the number of patients increases. Propose improvements for better usability and real-time updates.

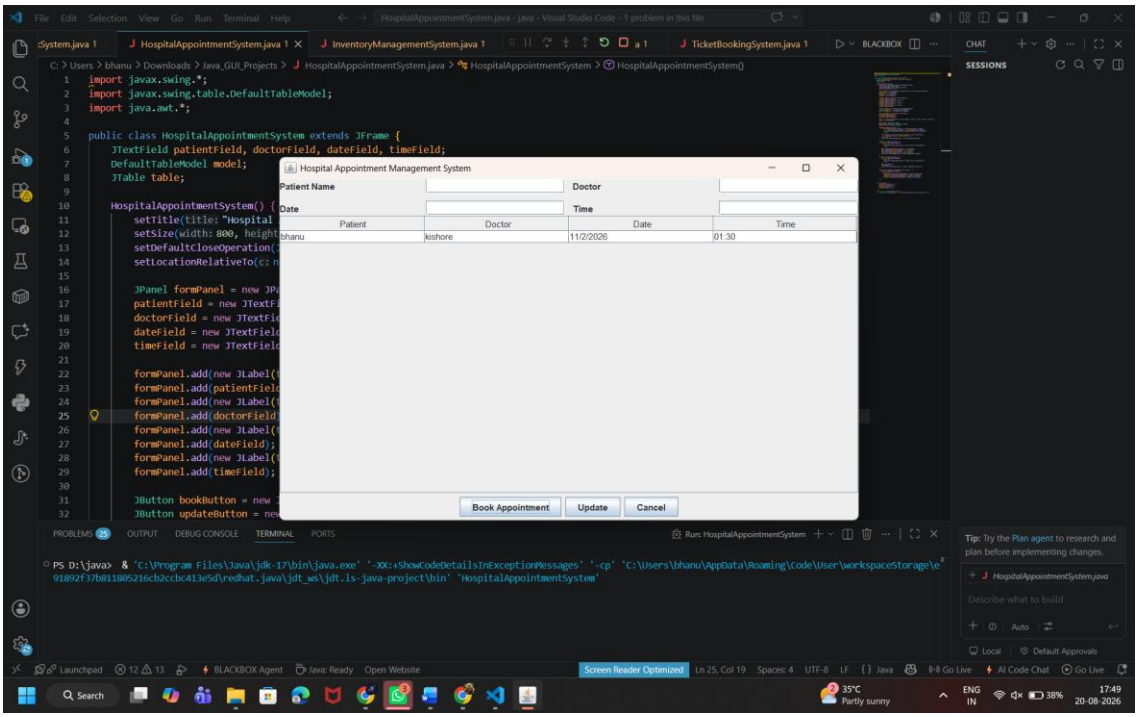
   Code:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class HospitalAppointmentSystem extends JFrame {
   JTextField patientField, doctorField, dateField, timeField;
   DefaultTableModel model;
   JTable table;

   HospitalAppointmentSystem() {
      setTitle("Hospital Appointment Management System");
      setSize(800, 500);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLocationRelativeTo(null);

      JPanel formPanel = new JPanel(new GridLayout(2, 4, 10, 10));
      patientField = new JTextField();
      doctorField = new JTextField();
      dateField = new JTextField();
      timeField = new JTextField();

      formPanel.add(new JLabel("Patient Name"));
      formPanel.add(patientField);
      formPanel.add(new JLabel("Doctor"));
      formPanel.add(doctorField);
      formPanel.add(new JLabel("Date"));
      formPanel.add(dateField);
      formPanel.add(new JLabel("Time"));
      formPanel.add(timeField);

      JButton bookButton = new JButton("Book Appointment");
      JButton updateButton = new JButton("Update");
```

```
        JButton cancelButton = new JButton("Cancel");

        JPanel buttonPanel = new JPanel(new FlowLayout());
        buttonPanel.add(bookButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(cancelButton);

         model = new DefaultTableModel(new String[]{"Patient", "Doctor", "Date", "Time"},
0);
        table = new JTable(model);

        add(formPanel, BorderLayout.NORTH);
        add(new JScrollPane(table), BorderLayout.CENTER);
        add(buttonPanel, BorderLayout.SOUTH);

        bookButton.addActionListener(e -> {
            if (patientField.getText().isEmpty() || doctorField.getText().isEmpty()
                    || dateField.getText().isEmpty() || timeField.getText().isEmpty()) {
                JOptionPane.showMessageDialog(this, "Please enter all details");
                return;
            }
            model.addRow(new Object[]{patientField.getText(), doctorField.getText(),
                    dateField.getText(), timeField.getText()});
            clearFields();
            JOptionPane.showMessageDialog(this, "Appointment Booked");
        });

        updateButton.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row == -1) {
                JOptionPane.showMessageDialog(this, "Select an appointment");
                return;
            }
            model.setValueAt(patientField.getText(), row, 0);
            model.setValueAt(doctorField.getText(), row, 1);
            model.setValueAt(dateField.getText(), row, 2);
            model.setValueAt(timeField.getText(), row, 3);
            JOptionPane.showMessageDialog(this, "Appointment Updated");
        });

        cancelButton.addActionListener(e -> {
            int row = table.getSelectedRow();
            if (row == -1) {
                JOptionPane.showMessageDialog(this, "Select an appointment");
```

```
            return;
        }
        model.removeRow(row);
        clearFields();
        JOptionPane.showMessageDialog(this, "Appointment Cancelled");
    });

    table.getSelectionModel().addListSelectionListener(e -> {
        int row = table.getSelectedRow();
        if (row != -1) {
            patientField.setText(model.getValueAt(row, 0).toString());
            doctorField.setText(model.getValueAt(row, 1).toString());
            dateField.setText(model.getValueAt(row, 2).toString());
            timeField.setText(model.getValueAt(row, 3).toString());
        }
    });
}

void clearFields() {
    patientField.setText("");
    doctorField.setText("");
    dateField.setText("");
    timeField.setText("");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new HospitalAppointmentSystem().setVisible(true));
}
}
```

Output:


Hospital Appointment Management System
Patient Name
Doctor
Date
Time
Patient
Doctor
Date
Time
bhanu
kishore
11/2/2026
01:30
Book Appointment
Update
Cancel

2. **Online Banking Interface**

Develop a GUI-based banking application that allows users to perform operations such as fund transfer, balance inquiry, and viewing transaction history. Use Swing components and event listeners to manage user interactions securely. Implement input validation for sensitive data like account number and amount. Ensure proper error handling for invalid transactions. Apply suitable layout managers to create a responsive and user-friendly interface. Analyze system responsiveness during multiple operations and suggest improvements. Demonstrate how secure input handling enhances reliability.

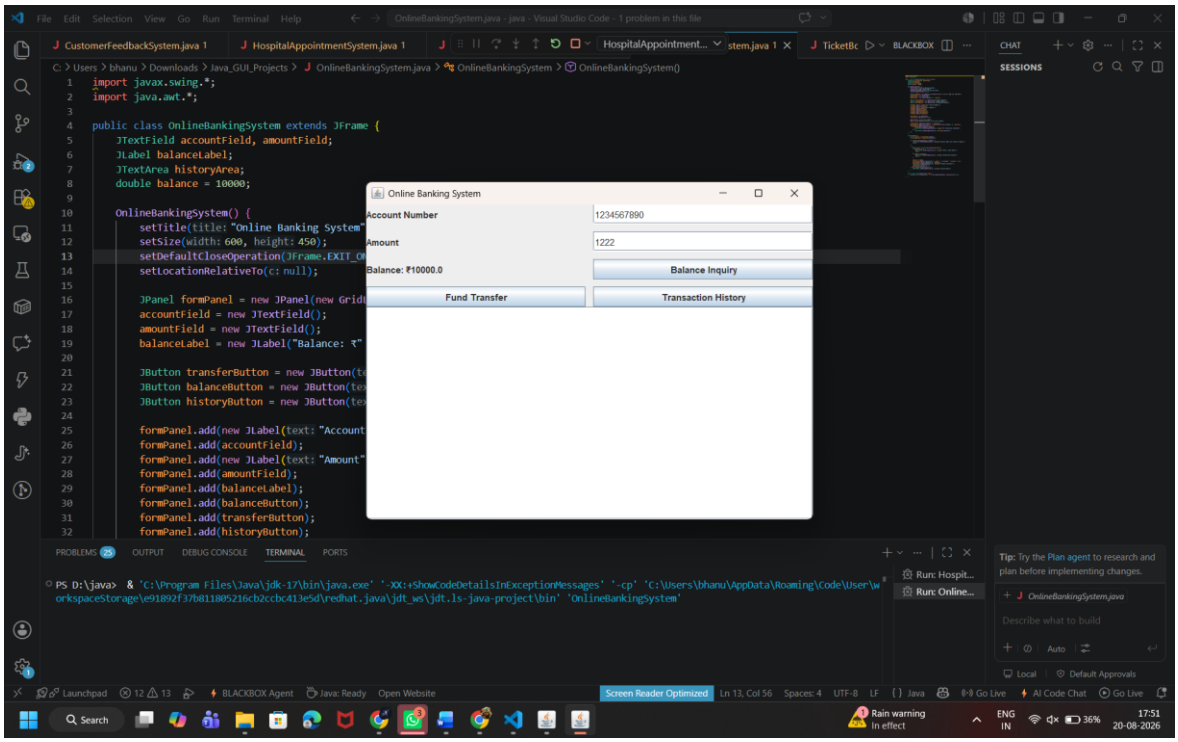
Code:

```
import javax.swing.*;
import java.awt.*;

public class OnlineBankingSystem extends JFrame {
   JTextField accountField, amountField;
   JLabel balanceLabel;
   JTextArea historyArea;
   double balance = 10000;

   OnlineBankingSystem() {
      setTitle("Online Banking System");
      setSize(600, 450);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLocationRelativeTo(null);

      JPanel formPanel = new JPanel(new GridLayout(4, 2, 10, 10));
      accountField = new JTextField();
      amountField = new JTextField();
      balanceLabel = new JLabel("Balance: ₹" + balance);

      JButton transferButton = new JButton("Fund Transfer");
      JButton balanceButton = new JButton("Balance Inquiry");
      JButton historyButton = new JButton("Transaction History");

      formPanel.add(new JLabel("Account Number"));
      formPanel.add(accountField);
      formPanel.add(new JLabel("Amount"));
      formPanel.add(amountField);
      formPanel.add(balanceLabel);
      formPanel.add(balanceButton);
      formPanel.add(transferButton);
      formPanel.add(historyButton);

      historyArea = new JTextArea();
```

```
        historyArea.setEditable(false);

        add(formPanel, BorderLayout.NORTH);
        add(new JScrollPane(historyArea), BorderLayout.CENTER);

        transferButton.addActionListener(e -> transfer());
        balanceButton.addActionListener(e -> balanceLabel.setText("Balance: ₹" + balance));
        historyButton.addActionListener(e -> {
            if (historyArea.getText().isEmpty())
                JOptionPane.showMessageDialog(this, "No transactions available");
            else
                JOptionPane.showMessageDialog(this, historyArea.getText());
        });
    }

    void transfer() {
        String account = accountField.getText();
        String amountText = amountField.getText();

        if (!account.matches("\\d{10}")) {
            JOptionPane.showMessageDialog(this, "Account number must contain 10 digits");
            return;
        }

        try {
            double amount = Double.parseDouble(amountText);
            if (amount <= 0) {
                JOptionPane.showMessageDialog(this, "Enter a valid amount");
                return;
            }
            if (amount > balance) {
                JOptionPane.showMessageDialog(this, "Insufficient balance");
                return;
            }

            balance -= amount;
            historyArea.append("Transferred ₹" + amount + " to Account " + account + "\n");
            balanceLabel.setText("Balance: ₹" + balance);
            JOptionPane.showMessageDialog(this, "Transfer Successful");
            accountField.setText("");
            amountField.setText("");
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Invalid amount");
        }
```

```
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new OnlineBankingSystem().setVisible(true));
    }
}
```

Output:



3. **Inventory Management Dashboard**

Create a GUI-based inventory management system using Swing components such as tables, forms, and menus to manage product details. Implement event handling for operations like adding, updating, and deleting products. Ensure efficient data display and manipulation for large inventories. Evaluate system scalability when handling thousands of records. Analyze usability aspects such as navigation, search functionality, and responsiveness. Suggest improvements like pagination or filtering for better performance. Justify your design choices based on real-world business requirements.

Code:

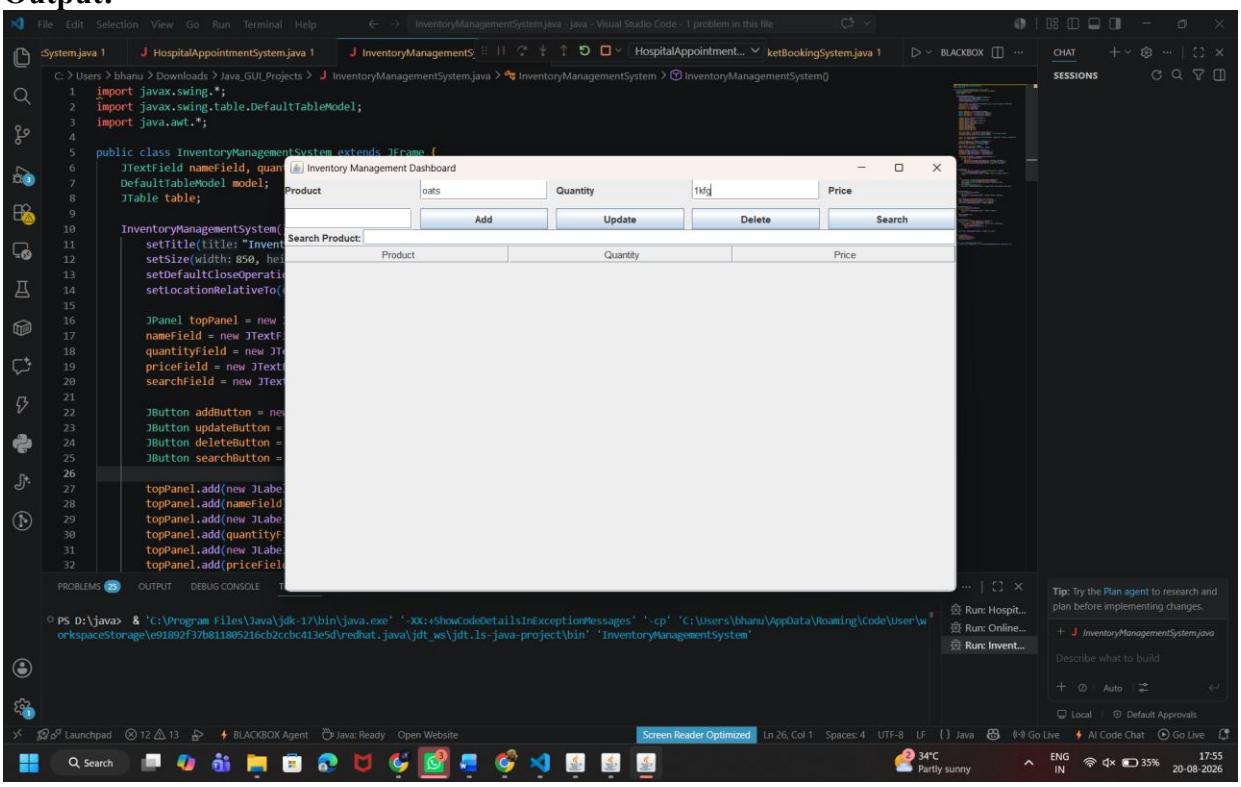
```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class InventoryManagementSystem extends JFrame {
   JTextField nameField, quantityField, priceField, searchField;
   DefaultTableModel model;
   JTable table;

   InventoryManagementSystem() {
      setTitle("Inventory Management Dashboard");
      setSize(850, 550);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLocationRelativeTo(null);

      JPanel topPanel = new JPanel(new GridLayout(2, 5, 10, 10));
      nameField = new JTextField();
      quantityField = new JTextField();
      priceField = new JTextField();
      searchField = new JTextField();

      JButton addButton = new JButton("Add");
      JButton updateButton = new JButton("Update");
      JButton deleteButton = new JButton("Delete");
      JButton searchButton = new JButton("Search");

      topPanel.add(new JLabel("Product"));
      topPanel.add(nameField);
      topPanel.add(new JLabel("Quantity"));
      topPanel.add(quantityField);
      topPanel.add(new JLabel("Price"));
      topPanel.add(priceField);
      topPanel.add(addButton);
      topPanel.add(updateButton);
      topPanel.add(deleteButton);
```

```
    topPanel.add(searchButton);

    JPanel searchPanel = new JPanel(new BorderLayout());
    searchPanel.add(new JLabel(" Search Product: "), BorderLayout.WEST);
    searchPanel.add(searchField, BorderLayout.CENTER);

    model = new DefaultTableModel(new String[]{"Product", "Quantity", "Price"}, 0);
    table = new JTable(model);

    JPanel northPanel = new JPanel(new BorderLayout());
    northPanel.add(topPanel, BorderLayout.CENTER);
    northPanel.add(searchPanel, BorderLayout.SOUTH);

    add(northPanel, BorderLayout.NORTH);
    add(new JScrollPane(table), BorderLayout.CENTER);

    addButton.addActionListener(e -> addProduct());
    updateButton.addActionListener(e -> updateProduct());
    deleteButton.addActionListener(e -> deleteProduct());
    searchButton.addActionListener(e -> searchProduct());

    table.getSelectionModel().addListSelectionListener(e -> {
        int row = table.getSelectedRow();
        if (row != -1) {
            nameField.setText(model.getValueAt(row, 0).toString());
            quantityField.setText(model.getValueAt(row, 1).toString());
            priceField.setText(model.getValueAt(row, 2).toString());
        }
    });
}

void addProduct() {
    if (nameField.getText().isEmpty() || quantityField.getText().isEmpty()
            || priceField.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Enter all product details");
        return;
    }

    try {
        int quantity = Integer.parseInt(quantityField.getText());
        double price = Double.parseDouble(priceField.getText());
        model.addRow(new Object[]{nameField.getText(), quantity, price});
        clearFields();
    } catch (NumberFormatException e) {
```

```
            JOptionPane.showMessageDialog(this, "Enter valid quantity and price");
        }
    }

    void updateProduct() {
        int row = table.getSelectedRow();
        if (row == -1) {
            JOptionPane.showMessageDialog(this, "Select a product");
            return;
        }
        model.setValueAt(nameField.getText(), row, 0);
        model.setValueAt(quantityField.getText(), row, 1);
        model.setValueAt(priceField.getText(), row, 2);
        JOptionPane.showMessageDialog(this, "Product Updated");
    }

    void deleteProduct() {
        int row = table.getSelectedRow();
        if (row == -1) {
            JOptionPane.showMessageDialog(this, "Select a product");
            return;
        }
        model.removeRow(row);
        clearFields();
    }

    void searchProduct() {
        String search = searchField.getText().toLowerCase();
        for (int i = 0; i < model.getRowCount(); i++) {
            String product = model.getValueAt(i, 0).toString().toLowerCase();
            if (product.contains(search)) {
                table.setRowSelectionInterval(i, i);
                return;
            }
        }
        JOptionPane.showMessageDialog(this, "Product not found");
    }

    void clearFields() {
        nameField.setText("");
        quantityField.setText("");
        priceField.setText("");
    }
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new InventoryManagementSystem().setVisible(true));
    }
}
```

Output:



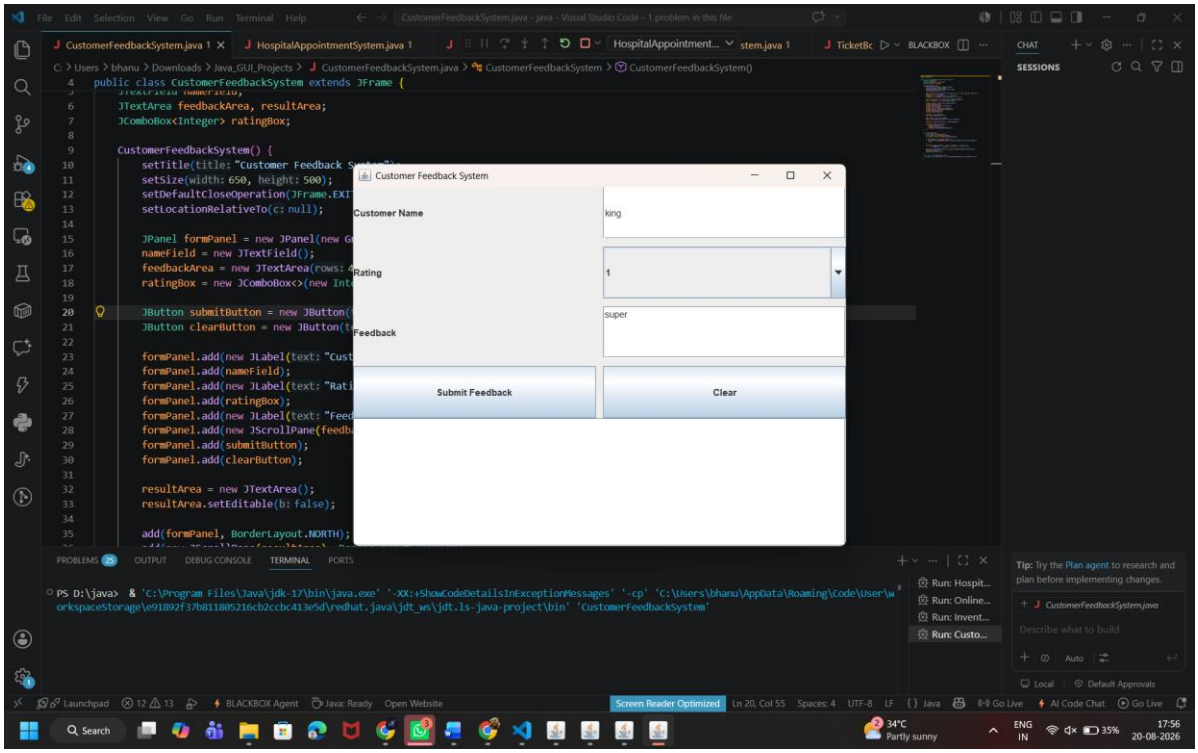
4. **Customer Feedback System**
Design an Applet-based GUI to collect customer feedback using input fields, rating options, and submission buttons. Implement event handling to process user inputs and dynamically update the display of feedback results. Ensure proper validation of inputs before submission. Analyze the limitations of Applet technology in modern applications, including security and browser compatibility issues. Evaluate the effectiveness of the chosen UI design and layout managers. Suggest modern alternatives such as web-based or desktop frameworks for improved performance and scalability.

Code:

```
import javax.swing.*;
import java.awt.*;

public class CustomerFeedbackSystem extends JFrame {
   JTextField nameField;
   JTextArea feedbackArea, resultArea;
   JComboBox<Integer> ratingBox;

   CustomerFeedbackSystem() {
      setTitle("Customer Feedback System");
      setSize(650, 500);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLocationRelativeTo(null);

      JPanel formPanel = new JPanel(new GridLayout(4, 2, 10, 10));
      nameField = new JTextField();
      feedbackArea = new JTextArea(4, 20);
      ratingBox = new JComboBox<>(new Integer[]{1, 2, 3, 4, 5});

      JButton submitButton = new JButton("Submit Feedback");
      JButton clearButton = new JButton("Clear");

      formPanel.add(new JLabel("Customer Name"));
      formPanel.add(nameField);
      formPanel.add(new JLabel("Rating"));
      formPanel.add(ratingBox);
      formPanel.add(new JLabel("Feedback"));
      formPanel.add(new JScrollPane(feedbackArea));
      formPanel.add(submitButton);
      formPanel.add(clearButton);

      resultArea = new JTextArea();
      resultArea.setEditable(false);

      add(formPanel, BorderLayout.NORTH);
      add(new JScrollPane(resultArea), BorderLayout.CENTER);

      submitButton.addActionListener(e -> submitFeedback());
```

```
        clearButton.addActionListener(e -> {
            nameField.setText("");
            feedbackArea.setText("");
            ratingBox.setSelectedIndex(0);
        });
    }

    void submitFeedback() {
        String name = nameField.getText();
        String feedback = feedbackArea.getText();
        int rating = (Integer) ratingBox.getSelectedItem();

        if (name.isEmpty() || feedback.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please enter name and feedback");
            return;
        }

        resultArea.append("Customer: " + name + "\nRating: " + rating + "/5"
                + "\nFeedback: " + feedback + "\n-------------------------\n");

        JOptionPane.showMessageDialog(this, "Feedback Submitted Successfully");
        nameField.setText("");
        feedbackArea.setText("");
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new CustomerFeedbackSystem().setVisible(true));
    }
}
```

Output:


Customer Feedback System
Customer Name
king
Rating
1
Feedback
super
Submit Feedback
Clear

5. Ticket Booking System

Develop a multi-window GUI application for ticket booking with features like seat selection, booking confirmation, and cancellation. Use Swing components such as buttons, panels, and menus along with appropriate layout managers to design the interface. Implement event handling to manage user interactions and ensure accurate seat allocation. Validate user inputs and handle exceptions during booking. Ensure smooth navigation between windows. Analyze user experience and identify areas for improving real-time interaction and responsiveness. Demonstrate the system with sample booking scenarios.

Code:

```
import javax.swing.*;
import java.awt.*;

public class TicketBookingSystem extends JFrame {
   JButton[] seats = new JButton[20];
   JTextField nameField;
   JLabel selectedLabel;
   int selectedSeat = -1;

   TicketBookingSystem() {
      setTitle("Ticket Booking System");
      setSize(700, 550);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLocationRelativeTo(null);

      JPanel topPanel = new JPanel(new FlowLayout());
      nameField = new JTextField(15);
      JButton bookButton = new JButton("Book Ticket");
      JButton cancelButton = new JButton("Cancel Booking");
      selectedLabel = new JLabel("Selected Seat: None");

      topPanel.add(new JLabel("Passenger Name:"));
      topPanel.add(nameField);
      topPanel.add(selectedLabel);

      JPanel seatPanel = new JPanel(new GridLayout(4, 5, 10, 10));

      for (int i = 0; i < 20; i++) {
         final int seatNumber = i + 1;
         seats[i] = new JButton("Seat " + seatNumber);
         seatPanel.add(seats[i]);

         seats[i].addActionListener(e -> {
            if (seats[seatNumber - 1].getText().startsWith("Booked")) {
               JOptionPane.showMessageDialog(this, "This seat is already booked");
               return;
            }
            selectedSeat = seatNumber;
```

```
            selectedLabel.setText("Selected Seat: " + seatNumber);
        });
    }

    JPanel bottomPanel = new JPanel();
    bottomPanel.add(bookButton);
    bottomPanel.add(cancelButton);

    add(topPanel, BorderLayout.NORTH);
    add(seatPanel, BorderLayout.CENTER);
    add(bottomPanel, BorderLayout.SOUTH);

    bookButton.addActionListener(e -> bookTicket());
    cancelButton.addActionListener(e -> cancelTicket());
}

void bookTicket() {
    if (nameField.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Enter passenger name");
        return;
    }

    if (selectedSeat == -1) {
        JOptionPane.showMessageDialog(this, "Select a seat");
        return;
    }

    seats[selectedSeat - 1].setText("Booked " + selectedSeat);
    seats[selectedSeat - 1].setEnabled(false);

    JOptionPane.showMessageDialog(this, "Ticket booked successfully\nPassenger: "
            + nameField.getText() + "\nSeat: " + selectedSeat);

    nameField.setText("");
    selectedSeat = -1;
    selectedLabel.setText("Selected Seat: None");
}

void cancelTicket() {
    String seatText = JOptionPane.showInputDialog(this, "Enter seat number to cancel:");

    try {
        int seatNumber = Integer.parseInt(seatText);

        if (seatNumber < 1 || seatNumber > 20) {
            JOptionPane.showMessageDialog(this, "Invalid seat number");
            return;
```

```
            }

            if (!seats[seatNumber - 1].getText().startsWith("Booked")) {
                JOptionPane.showMessageDialog(this, "Seat is not booked");
                return;
            }

            seats[seatNumber - 1].setText("Seat " + seatNumber);
            seats[seatNumber - 1].setEnabled(true);
            JOptionPane.showMessageDialog(this, "Booking cancelled");

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Enter a valid seat number");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new TicketBookingSystem().setVisible(true));
    }
}
```

Output: